ECE 276A Project 2 Report

Nathan Van Utrecht
Electrical and Computer Engineering
University of California San Diego
San Diego, California
nvanutrecht@ucsd.edu

***Abstract*—This report details the implementation of a Simultaneous Localization and Mapping (SLAM) pipeline for a differential-drive robot using data from wheel encoders, an IMU, a 2D LiDAR scanner, and an RGBD camera. The pipeline begins with dead-reckoning odometry, refined through Iterative Closest Point (ICP) LiDAR scan matching. To construct globally consistent 2D occupancy and texture maps, the trajectory is further optimized using Georgia Tech Smoothing and Mapping (GTSAM) via a pose graph optimization framework. Two loop-closure detection strategies—fixed-interval and proximity-based—are compared, demonstrating that the proximity-based approach effectively mitigates cumulative drift, resulting in high-fidelity mapping of the robot's environment.**

I. Introduction

Autonomous navigation requires a robot to accurately comprehend its surroundings while simultaneously tracking its own position within them. Simultaneous localization and mapping (SLAM) addresses this challenge and is an essential foundation for exploring unknown environments in robotic applications. In this project, I implement SLAM for a differential-drive robot. My primary objective is to use sensor measurements to localize the robot, build a 2-D occupancy grid map of the environment, and then also create a texture map of the floor using RGBD imagery.

To achieve this, my general approach involves processing data streams from the robot's encoders, IMU, LiDAT, and RGBD camera. I first estimate the robot's trajectory, then refine it through LiDAR scan matching and factor graph optimization with loop-closure constraints. Finally, I utilize these optimized trajectory estimates to construct the final occupancy and texture maps.

II. Problem Formulation

The overarching goal of this project is to estimate the trajectory of a differential-drive robot and construct a 2-D occupancy grid and texture map of its environment. Let the robot's pose at time t be defined in the world frame as $\mathbf{x}_t = [x_t, y_t, \theta_t]^T \in SE(2)$, where x_t and y_t represent the 2-D position and θ_t represents the orientation. The trajectory over T time steps is denoted as $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_T\}$.

A. Odometry Kinematics

The robot provides control inputs $\mathbf{u}_t = [v_t, \omega_t]^T$, where v_t is the linear velocity derived from the wheel encoders and ω_t is the yaw rate provided by the IMU. Assuming an initial identity pose, the odometry problem is to determine the subsequent pose $\mathbf{x}_{t+1}$ by applying a discrete-time differential-drive motion model:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) \tag{1}$$

B. LiDAR Scan Matching

At each time step t, the LiDAR outputs a scan S_t consisting of range measurements that are projected into a 2-D local point cloud. Let the target point set (the previous scan) be $\{\mathbf{m}_i\}$ and the source point set (the current scan) be $\{\mathbf{z}_j\}$. Because the data association Δ between the points is unknown (i.e. the first point of the previous scan might not correlate to the first point of the current scan), the scan matching problem seeks to iteratively find the optimal correspondences and the relative 2-D rigid-body transformation (rotation R and translation $\mathbf{p}$) that minimizes the squared Euclidean distance between associated points:

$$\min_{R \in SO(2), \mathbf{p}} \sum_{i,j \in \Delta} ||\mathbf{m}_i - (R\mathbf{z}_j + \mathbf{p})||_2^2 \tag{2}$$

This transformation ${}_tT_{t+1}$ is then used to estimate the new robot pose:

$$T_{t+1} = T_t *_t T_{t+1} \tag{3}$$

C. Occupancy Grid

Given the estimated trajectory $\mathcal{X}$ and the sequence of LiDAR scans $\mathcal{S} = \{S_1, ..., S_T\}$, the mapping objective is to estimate a 2-D occupancy grid map M. The map is discretized into cells m_i, where the objective is to compute the log-odds probability of each cell being occupied given the history of poses and measurements:

$$P(m_i \mid \mathcal{X}, \mathcal{S}) \tag{4}$$

This requires mapping the physical LiDAR points from the sensor frame into the world frame to identify which discrete cells are being occupied and which are free space.

D. Texture Mapping

In addition to the physical layout, the robot also generates a 2-D texture map C of the floor. Given depth images and RGB images from the Kinect sensor, the problem involves finding a mathematical projection from the 2-D pixel coordinates (u, v) and disparity d to a 3-D point in the world frame. By isolating the 3-D points that intersect the floor plane ($z \approx 0$), the goal is to assign the corresponding RGB color value to the correct cell in the grid map.

E. Pose Graph Optimization

To correct cumulative drift, I formulate a pose graph optimization problem. Let the factor graph be defined by a set of vertices $\mathcal{V}$ representing the unknown robot poses $\mathbf{T}_i \in SE(2)$ to be estimated, and a set of edges $\mathcal{E}$ representing the measurement constraints. The measurements, denoted as $\bar{\mathbf{T}}_{ij}$, are the relative poses obtained from the ICP scan matching odometry and loop closure between poses i and j.

Because the robot poses are rigid body transformations, the difference between variables must respecct their specific space geometry. The relative pose error vector $\mathbf{e}(\mathbf{T}_i, \mathbf{T}_j)$ is defined using the logarithmic map:

$$\mathbf{e}(\mathbf{T}_i, \mathbf{T}_j) = \log(\bar{\mathbf{T}}_{ij}^{-1}\mathbf{T}_i^{-1}\mathbf{T}_j^{-1})^\vee \tag{5}$$

Here, $\vee$ represents the hat map. The optimization problem seeks to find the optimal set of poses $\{\mathbf{T}_i\}$ that minimizes the sum of squared, weighted residual errors across all edges in the pose graph:

$$\min_{\{\mathbf{T}_i\}} \sum_{i,j} \in \mathcal{E} ||\mathbf{W}_{ij} \log(\bar{\mathbf{T}}_{ij}^{-1}\mathbf{T}_i^{-1}\mathbf{T}_j^{-1})^\vee||_2^2 \tag{6}$$

where $\mathbf{W}_{ij}$ represents the weight matrix associated with the certainty of each relative pose measurement.

III. Technical Approach

A. Odometry Kinematics

To establish an initial baseline trajectory for the robot, I implemented a dead-reckoning approach using a differential-drive motion model. This required fusing the linear velocity derived from the wheel encoders with the angular velocity provided by the IMU.

1) Velocity Calculation: The encoders report the cumulative ticks for the front-right (FR), front-left (FL), rear-right (RR), and rear-left (RL) wheels at 40 Hz. I calculated the distance traveled by the right wheels as $(FR+RR)/2\times 0.0022$ meters, and the left wheels as $(FL+RL)/2\times 0.0022$ meters. The average of the left and right distances over a given time interval Δt yielded the overall linear velocity of the robot, v_t.

2) Temporal Sensor Synchronization: Because the encoders and the IMU operate asynchronously at different frequencies, I had to align their measurements. I chose the encoder timestamps as the base frequency. For each encoder time interval $[t_{k-1}, t_k]$, I identified all corresponding IMU readings that fell within that window. I numerically integrated the raw IMU yaw rate (the z-axis angular velocity) over these discrete subintervals to compute the total change in heading ($\Delta\theta$) for that specific encoder step. The effective yaw rate for the interval was then computed as $\omega_t = \Delta\theta/\Delta t$.

3) Motion Model Integration: With the synchronized control inputs v_t and ω_t, I updated the robot's pose $\mathbf{x}_t = [x_t, y_t, \theta_t]^T$. To accurately integrate the trajectory and avoid numerical instability when the robot travels in a straight line ($\omega_t \approx 0$), I utilized an exact analytical solution for the discrete-time differential-drive motion model:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \Delta t \begin{bmatrix} v_t \operatorname{sinc}\left(\frac{\omega_t \Delta t}{2}\right) \cos\left(\theta_t + \frac{\omega_t \Delta t}{2}\right) \\ v_t \operatorname{sinc}\left(\frac{\omega_t \Delta t}{2}\right) \sin\left(\theta_t + \frac{\omega_t \Delta t}{2}\right) \\ \omega_t \end{bmatrix} \tag{7}$$

Iterating this model over the entire dataset generated the initial unoptimized odometry trajectory.

B. LiDAR Scan Matching

To refine the dead-reckoning trajectory and correct for sensor drift, I implemented an Iterative Closest Point (ICP) algorithm to estimate the relative 2D transformations between consecutive LiDAR scans.

1) Preprocessing and Initialization: First, I converted the raw polar LiDAR measurements into 2D cartesian coordinates. I then filtered out invalid points outside the sensor's valid range ([0.1m, 30m]). Finally I transformed them into the robot's body frame using the provided CAD schematic for measurements. Because ICP is an optimization algorithm that readily converges to local optima, providing a strong initial guess is critical. For each pair of consecutive scans, I initialized the relative transformation by integrating the encoder and IMU odometry over the exact time window between the two LiDAR timestamps.

2) Matching and Outlier Rejection: During each iteration of the ICP algorithm, I utilized a *k-d tree* to efficiently search for the nearest neighbors between the transformed source point cloud and the target point cloud. To ensure the optimization was robust to noise and dynamic obstacles, I applied a dynamic outlier rejection step. Point pairs with a Euclidean distance greater than three times the median distance of all matches were classified as outliers and discarded.

3) Optimal Alignment via Kabsch Algorithm: Given the filtered set of point correspondences Δ, I isolated the rotation from the translation by computing the centroids of the source points $\bar{\mathbf{z}}$ and target points $\bar{\mathbf{m}}$, and subtracting them to obtain the mean-centered point sets, $\tilde{\mathbf{z}}_j$ and $\tilde{\mathbf{m}}_i$. I then constructed the cross-covariance matrix Q:

$$Q = \sum_{i,j\in\Delta} \tilde{\mathbf{m}}_i \tilde{\mathbf{z}}_j^T \tag{8}$$

To solve Wahba's problem and find the optimal rotation, I performed Singular Value Decomposition (SVD) on this matrix such that $Q = U\Sigma V^T$. To ensure the resulting matrix R was a valid rotation and to avoid reflections, I computed it as:

$$R = U \begin{bmatrix} 1 & 0 \\ 0 & \det(UV^T) \end{bmatrix} V^T \tag{9}$$

Finally, the optimal translation vector $\mathbf{p}$ was recovered by aligning the rotated source centroid with the target centroid:

$$\mathbf{p} = \bar{\mathbf{m}} - R\bar{\mathbf{z}} \tag{10}$$

4) Iteration and Trajectory Accumulation: The nearest-neighbor association and Kabsch algorithm steps were repeated iteratively until the change in the estimated transformation fell below a strict tolerance threshold (10^{-4}) or reached the maximum limit of 50 iterations. Once converged, the resulting relative transformation ${}_tT_{t+1}$ was plugged into Equation 3 to get the updated robot pose T_{t+1}.

5) 3D Point-Cloud Registration Warm-up: Prior to executing the 2D scan matching on the robot trajectory, I validated my core ICP methodology by implementing a 3D point-cloud registration warm-up. This task involved estimating the spatial poses of a drill and liquid container from depth images using canonical object models. To overcome initialization challenges in 3D space, I discretely sampled initial yaw angles around the z-axis and selected the final transformation that yielded the lowest mean squared error among the correspondences.

C. Occupancy Grid

With the robot trajectory estimated, I constructed a 2-D occupancy grid map of the environment. I initialized a discrete grid with a resolution of 0.05 meters per cell, spanning from -30.0m to 30.0m in both the *x* and *y* directions. The state of each cell was tracked using log-odds probabilities to efficiently accumulate evidence over time.

1) Point Transformation: For each time step, I transformed the valid 2-D LiDAR hits from the sensor frame into the robot's body frame using the static LiDAR to body transformation calculated from the provided CAD schematic. This was then subsequently converted into the global world frame using the robot's estimated pose $\mathbf{x}_t$. These global coordinates were then discretized into grid cell indices.

2) Log-Odds Update via Polygon Filling: To identify the free space between the sensor and the obstacles, I utilized an efficient polygon-filling approach. By treating the sensor's origin and the ordered sequence of LiDAR hit points as the vertices of a polygon, I applied OpenCV's `fillPoly` function to instantly mask all intermediate cells. To account for sensor noise, the map was updated using a recursive Bayesian filter in log-odds form. Let $P(m_i \mid \mathbf{z}_{1:t}, \mathbf{x}_{1:t})$ be the probability that a grid cell m_i is occupied given the history of measurements $\mathbf{z}_{1:t}$ and poses $\mathbf{x}_{1:t}$. The log-odds representation $L_t(m_i)$ is defined as:

$$L_t(m_i) = \log\left(\frac{P(m_i \mid \mathbf{z}_{1:t}, \mathbf{x}_{1:t})}{1 - P(m_i \mid \mathbf{z}_{1:t}, \mathbf{x}_{1:t})}\right) \tag{11}$$

Assuming a uniform prior probability of 0.5 for unknown cells (where $L_0(m_i) = \log(0.5/0.5) = 0$), the recursive update rule simplifies to the addition of the sensor model at time t:

$$L_t(m_i) = L_{t-1}(m_i) + \log\left(\frac{P(m_i \mid \mathbf{z}_t, \mathbf{x}_t)}{1 - P(m_i \mid \mathbf{z}_t, \mathbf{x}_t)}\right) \tag{12}$$

I manually picked to trust the sensor 80% of the time which results in the update terms $L_{occ} = \log(4)$ and $L_{free} = \log(1/4)$. To prevent absolute certainty and numerical instability over repeated observations, the log-odds values within the map M were clipped at each time step:

$$L_t(m_i) = \max(-10, \min(10, L_t(m_i)) \tag{13}$$

D. Pose Graph Optimization and Loop Closure

To globally optimize the estimated trajectory and mitigate accumulated drift, I constructed a 2-D factor graph using the Georgia Tech Smoothing and Mapping (GTSAM) library.

1) Factor Graph Construction: Let the full state trajectory be denoted as $\mathcal{T} = \{\mathbf{T}_0, \mathbf{T}_1, ..., \mathbf{T}_{n-1}\}$ where each node $\mathbf{T}_i \in SE(2)$ represents an unknown robot pose to be estimated. I anchored the graph to the world frame by applying a prior factor to the initial node $\mathbf{T}_0$ parameterized by a covariance matrix Σ_{prior}.

To create a constrained sequential backbone, I introduced two parallel `BetweenFactorPose2` edges between every consecutive pair of nodes $(\mathbf{T}_{i-1}, \mathbf{T}_i)$. The first edge enforces the relative pose measurement derived from the IMU and encoder odometry, denoted $\bar{\mathbf{T}}^{odom}_{i-1,i}$, with an associated measurement covariance Σ_{odom}. The second edge enforces the relative pose change computed by the 2-D ICP scan matching, $\bar{\mathbf{T}}^{icp}_{i-1,i}$, with covariance Σ_{icp}.

2) Loop Closure Detection: Let $\mathcal{L}$ represent the set of identified non-sequential loop closure constraints (j, i) between historical poses j and current poses i. To populate $\mathcal{L}$, I implemented two distinct search strategies:

- **Fixed-Interval Loop Closures:** I evaluated potential closures every 10 scans.
- **Proximity-Based Loop Closures:** I queried historical poses $\mathbf{T}_j$ whose Euclidean distance from the current pose $\mathbf{T}_i$ was less than a threshold of 0.8m, provided a minimum temporal gap of 200 timesteps had elapsed.

For any candidate pair (i, j), I initialized the ICP alignment using the inverse composition of the current global pose estimates:

$$\mathbf{T}_{init} = \mathbf{T}_j^{-1}\mathbf{T}_i \tag{14}$$

To ensure the graph optimization remained robust against false positives, I implemented a consistency check. A candidate loop-closure edge $\bar{\mathbf{T}}^{lc}_{j,i}$ was only added to $\mathcal{L}$ if its ICP fitness score f (defined as the mean Euclidean distance of the inlier point correspondences) fell below 0.1 m.

3) Optimization and Error Formulation: The back-end graph optimization problem seeks to find the MAP trajectory estimate $\mathcal{X}^*$. Since loop closures can introduce distortions if they are incorrect, I wrapped the loop-closure noise models Σ_{lc} in a Cauchy M-estimator, denoted by the loss function $\rho(\cdot)$, to aggressively discount extreme outliers.

The relative pose error vector is defined using Equation 5. The total cost function minimized during the optimization is the sum of the squared Mahalanobis distance of all factor residuals. This non-linear least-squares problem was solved iteratively using the Levenberg-Marquardt algorithm. The resulting optimized poses were extracted and fed back into the mapping pipeline to reconstruct the final occupancy grid and texture maps.

IV. Results

I evaluated my SLAM pipeline on both datasets, but the figures referenced in this section are from dataset 20. The rest of the results can be found in the Appendix.

A. ICP Warm-Up: 3D Object Pose Estimation

To validate my ICP implementation before applying it to the SLAM pipeline, 3D point cloud registration was performed on a drill and a liquid container.

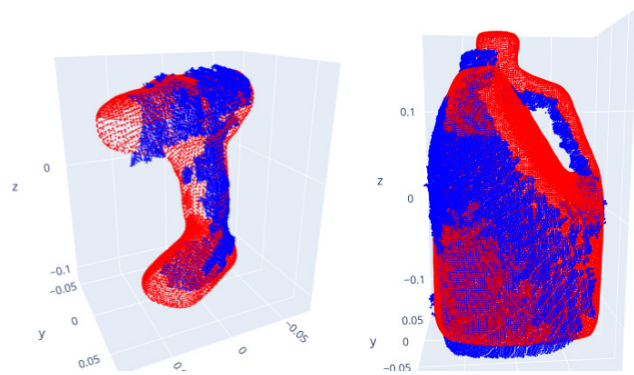

Fig. 1: ICP warmup results on the provided 0 dataset for the drill and liquid container objects. The rest of the results can be found in the Appendix.

Figure 1 illustrates the successful alignment of partial-view point clouds to their canonical models. This aligned drill point cloud demonstrates a tight overlap between the source and the canonical model with the overall geometry faithfully recovered. The liquid container achieves similar alignment, but regions near the container's handle exhibit slightly higher errors. This discrepancy is a direct result of self-occlusion in the source scan which limits the available correspondences in that localized area. Overall, these results confirmed that the ICP implementation converges reliably and can be used as a foundation for 2D scan matching.

B. Odometry Kinematics

As shown in Figure 2, the open-loop nature of this integration yields a trajectory that is only qualitatively reasonable over short distances. Over the full dataset, the trajectory exhibits severe cumulative drift. The robot's path fails to close loops, and the final estimated endpoint deviates massively from the starting area. This physical divergence highlights the limitations of relying solely on kinematic dead reckoning.

C. LiDAR Scan Matching

Figure 3 overlays the trajectory generated by 2D ICP scan matching against the baseline dead-reckoning trajectory.

The scan matching trajectory demonstrates an improvement in local consistency. While the robot drives down the rectilinear hallway, the scan matching maintains straight lines while the dead reckoning curved arbitrarily. It also exhibits significantly reduced drift in regions where the robot loops back on itself.

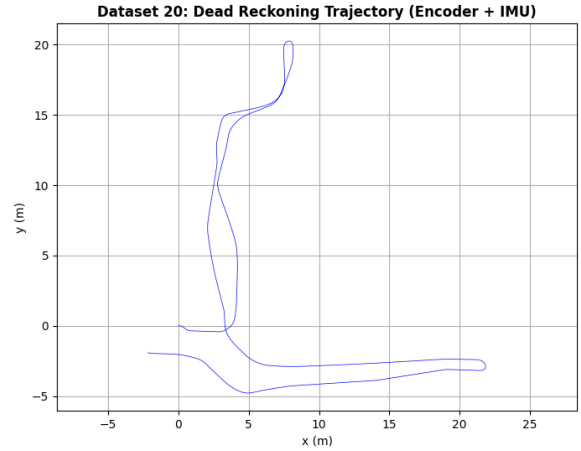

Fig. 2: Initial dead-reckoning trajectory for Dataset 20, generated purely from wheel encoder velocities and IMU yaw rates. The open-loop integration exhibits severe cumulative drift over time, causing the robot's estimated path to diverge and fail to close loops.

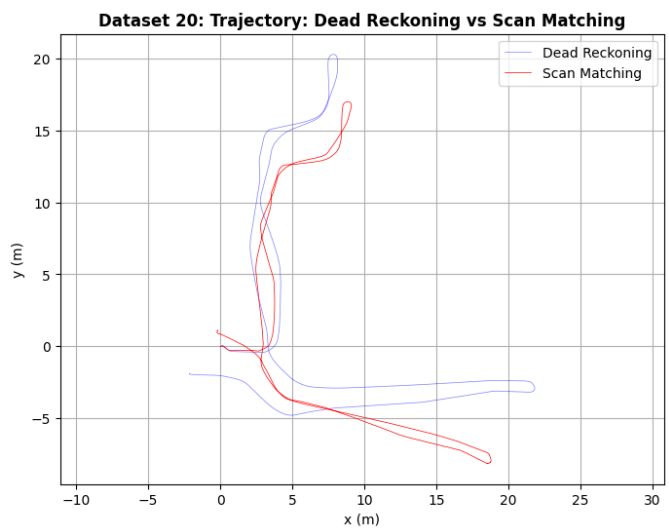

Fig. 3: Comparison of the estimated robot trajectories generated by dead-reckoning odometry and 2D ICP LiDAR scan matching for Dataset 20. The scan-matching trajectory demonstrates improved local consistency and maintains rectilinear paths, though an anomaly is visible at a corner junction where the ICP algorithm temporarily converged to a local minimum.

However, a notable anomaly occurred when the robot navigated a presumed corner of the hallway. At this junction, the ICP algorithm temporarily lost its geometric lock. Because the structural features of the corner lacked sufficient distinctiveness, the data registration algorithm converged to a local minimum. This caused the estimated trajectory to artificially distort the angle of the turn.

Because this approach relies strictly on frame-to-frame

$SE(2)$ relative transforms, this localized confusion at the corner compounded the residual drift that accumulates over long traversal.

D. Occupancy Grid and Texture Mapping

Using the estimated trajectories, 2D occupancy grids and floor texture maps were constructed to evaluate the impact of local trajectory correction. Maps generated strictly from encoder-IMU odometry, shown in the bottom row of Figure 4, exhibit severe cumulative drift. In the occupancy grid, this manifests as significant "ghosting," where walls and obstacles are drawn multiple times in progressively offset positions. This results in artificially thick, blurry boundaries and a main hallway that incorrectly curves. The texture map corroborates this drift, as the robot's outward and return paths visibly diverge into two separate tracks rather than overlapping on the same physical floor space.

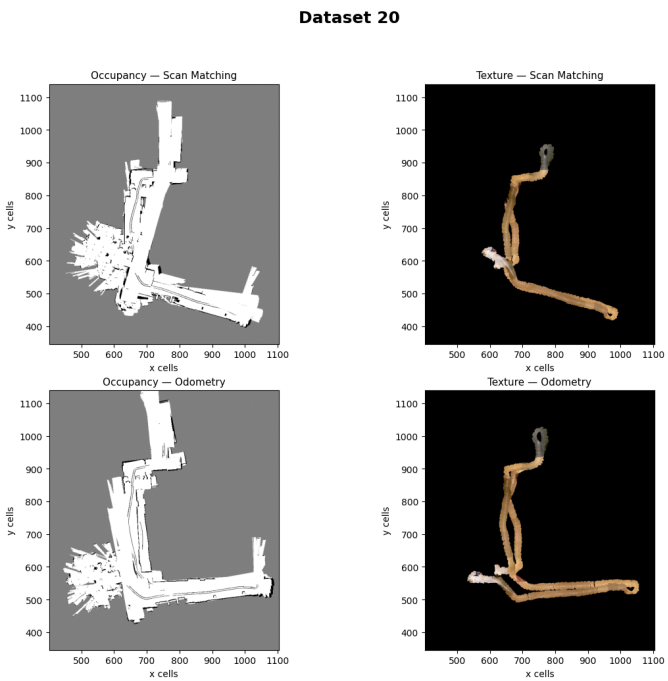

Fig. 4: 2D occupancy grids and floor texture maps generated prior to global optimization. The bottom row displays maps constructed solely from unoptimized encoder-IMU odometry, exhibiting severe cumulative drift and structural ghosting. The top row shows maps generated using 2D ICP relative transforms, which vastly improve boundary definition but still retain minor residual drift in the bottom-left region.

Conversely, applying the 2D ICP relative transforms yields a marked improvement in geometric consistency, as seen in the scan-matching maps in the top row. The occupancy grid features much sharper, highly defined walls, and the rectilinear structure of the main hallways is successfully preserved. Similarly, the texture map demonstrates much tighter spatial cohesion, with the outward and return paths overlapping cleanly.

However, while scan matching drastically improves the map quality, it does not achieve perfect global consistency. Subtle ghosting and structural misalignments remain visible in the scan-matching occupancy map, particularly in the complex, clutter-heavy region in the bottom left. This residual drift confirms that while frame-to-frame scan matching effectively limits local errors, a global correction mechanism is still required to produce a structurally perfect map.

E. Pose Graph Optimization

The quantitative differences between the fixed interval strategy and proximity based strategy can be seen in Table **??**

TABLE I: Comparison of Loop Closure Strategies

Strategy	Loop Closures	Factors	Initial Error	Final Error	Reduction
Fixed-Interval	496	10,419	5.04	2.87	43.1%
Proximity-Based	876	10,799	682.61	236.88	65.3%

The proximity based strategy exhibits a massive initial error compared to the fixed interval strategy. This occurs because proximity based loop closures link temporally distant poses, introducing rigid spatial constraints that heavily conflict with the accumulated drift in the unoptimized trajectory. However, the optimizer successfully resolves these conflicts, yielding a superior 65.3% reduction in total graph error. Conversely, the Fixed-Interval strategy introduces short-range constraints that are already relatively consistent with the local trajectory, resulting in a much lower initial error but a less impactful overall correction.

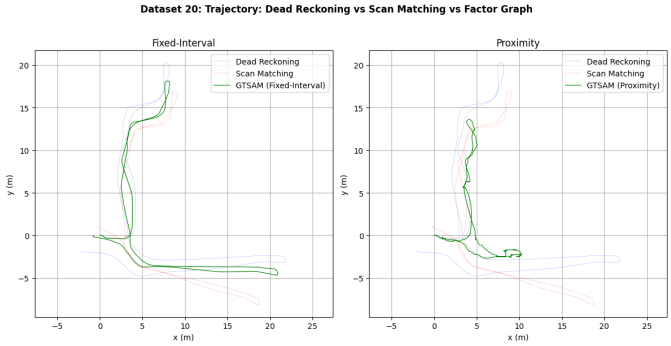

Fig. 5: Final trajectory comparison following pose graph optimization. The plot contrasts the unoptimized scan-matching trajectory with the optimized paths generated via fixed-interval and proximity-based loop closure strategies. The proximity-based approach yields the most compact and loop-consistent trajectory, successfully mitigating accumulated global drift.

Figure 5 visually confirms this data by comparing the trajectories. While both optimized trajectories improve upon the raw scan-matching path, the proximity based trajectory is noticeably more compact and loop-consistent, effectively snapping the drifted regions back into alignment.

1) SLAM Progression: Figures 6 and 7 illustrate the progressive map refinement for the fixed interval and proximity based strategies respectively across four checkpoints. For

the fixed interval strategy, the corrections are subtle. While the map shows progressively sharper wall boundaries, the overall trajectory largely tracks the original scan-matching path. Conversely, the proximity-based strategy demonstrates dramatic trajectory corrections, particularly at the 75% and 100% checkpoints when the robot re-enters previously mapped territory. Notably, at the third checkpoint, a highly visible map correction occurs as the optimizer pulls the current trajectory into alignment with historical spatial constraints. Consequently, the final occupancy and texture maps produced by this proximity-based strategy exhibit the highest visual fidelity.

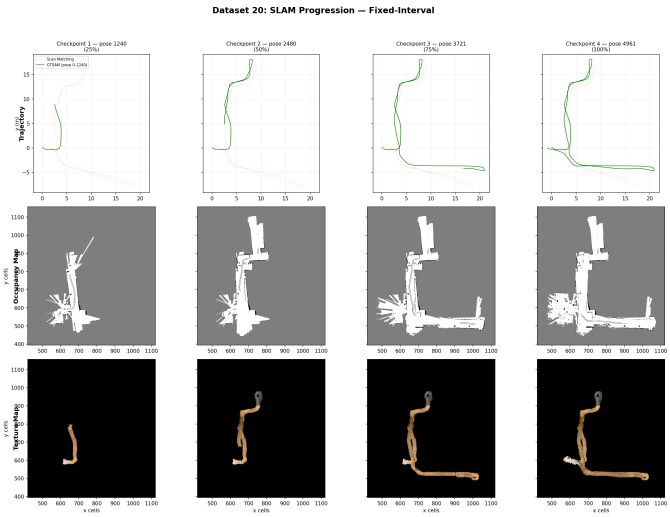

Fig. 6: Progressive map refinement utilizing the fixed-interval loop closure strategy evaluated across four trajectory checkpoints. The global structural adjustments are subtle, yielding sharper wall boundaries while the overall trajectory largely tracks the original scan-matching path.

F. Discussion

The results clearly demonstrate that while scan matching improves local trajectory accuracy compared to dead reckoning, it is insufficient for global map consistency due to drift.

The factor graph formulation successfully mitigates this drift, but the quality of the final map is highly dependent on the loop closure strategy. The proximity based approach proved superior because it directly targets the fundamental failure mode of SLAM which is correcting drift when revisiting a known spatial region. Furthermore, the successful convergence of the proximity graph highlights the importance of the Cauchy M-estimator which prevented erroneous ICP loop closures from distorting the global map.

While accurate, the current reliance on pairwise ICP for all loop closure verifications is computationally heavy. Future iterations of this pipeline would benefit from integrating other information to rapidly filter candidate priors to geometric verification.

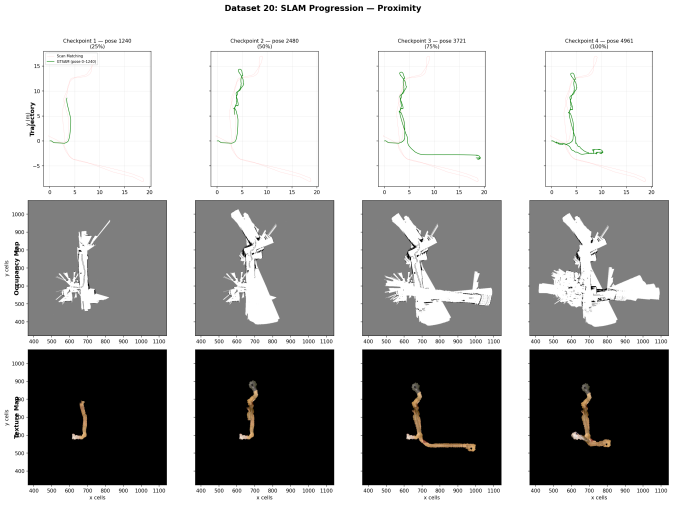

Fig. 7: Progressive map refinement utilizing the proximity-based loop closure strategy evaluated across four trajectory checkpoints. Dramatic trajectory corrections are visible—particularly at the 75% and 100% checkpoints—as the optimizer aggressively pulls the drifted trajectory back into alignment with historical spatial constraints, producing a high-fidelity final map.

V. Appendix

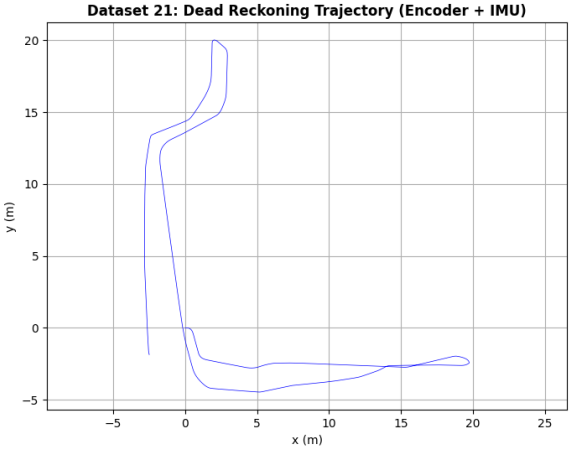

Fig. 8: Dead reckoning trajectory for dataset 21.

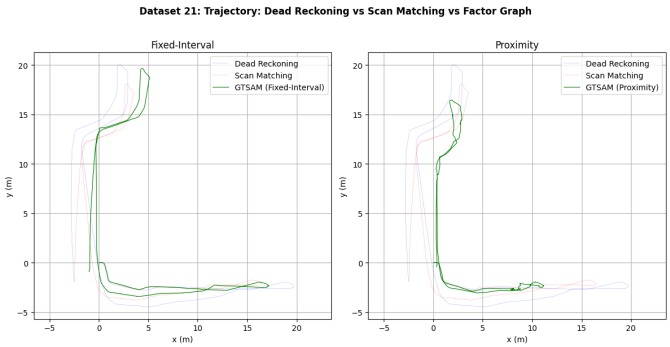

Fig. 9: Factor graph optimized trajectory for dataset 21.

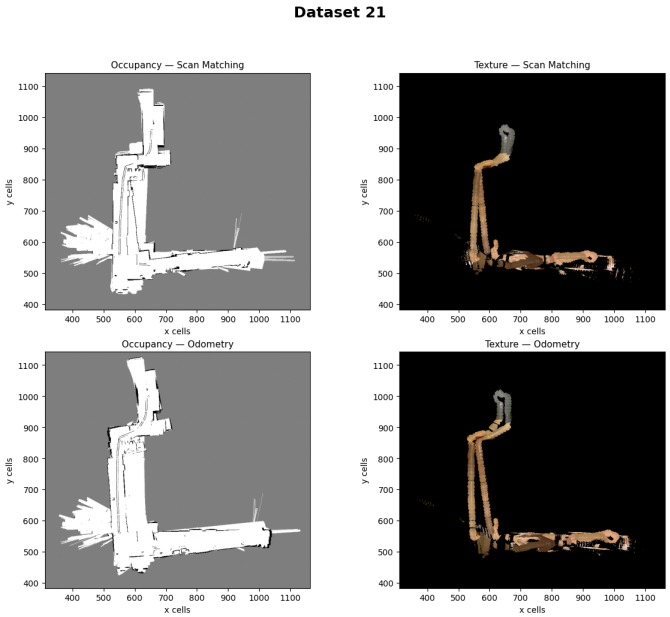

Fig. 10: Occupancy grid generated from unoptimized odometry for dataset 21.

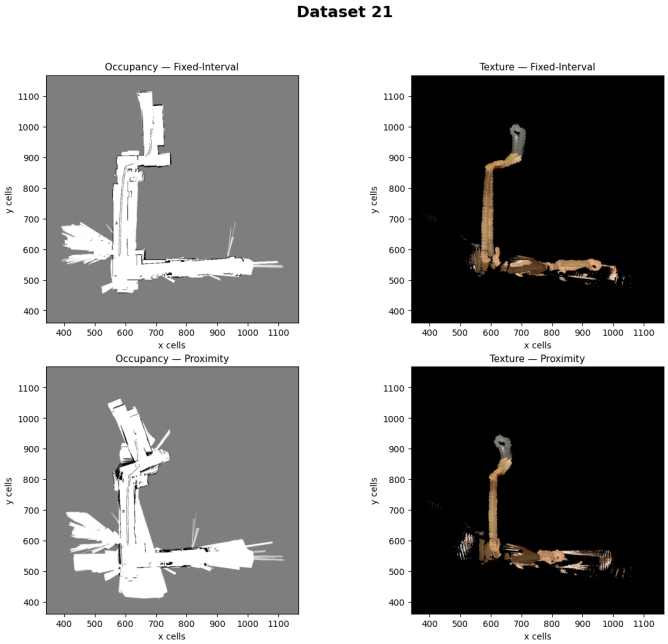

Fig. 11: Occupancy and texture maps for dataset 21.

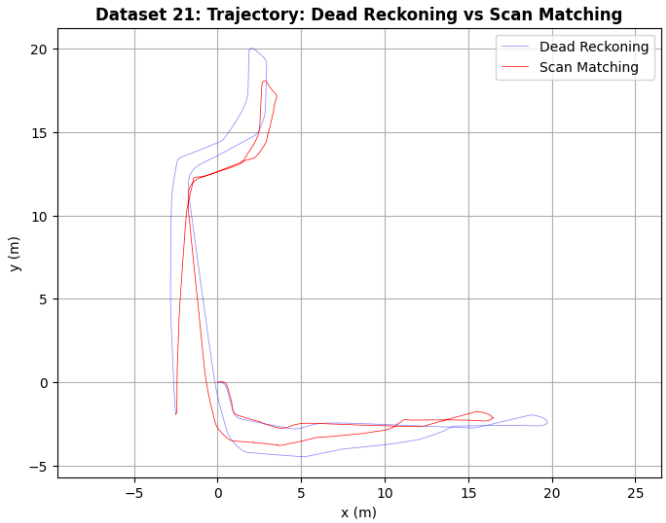

Fig. 12: Trajectory estimated via 2D ICP LiDAR scan matching for dataset 21.

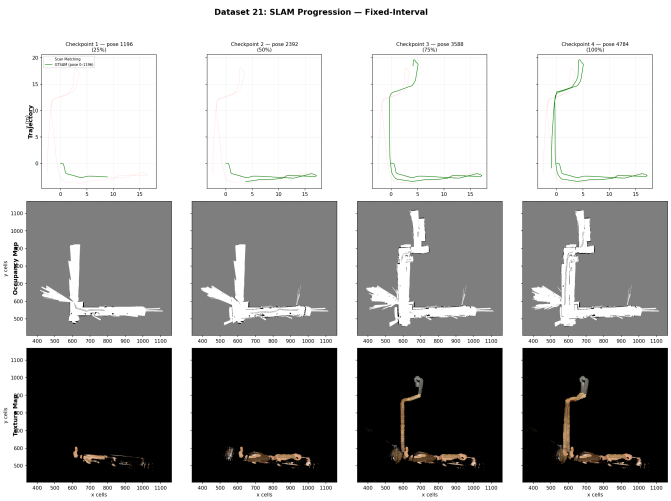

Fig. 13: SLAM map refinement progression using the fixed-interval loop closure strategy.

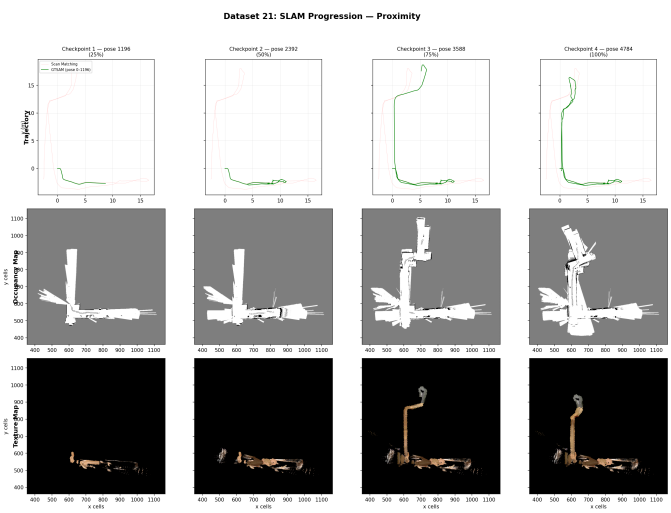

Fig. 14: SLAM map refinement progression using the proximity-based loop closure strategy.

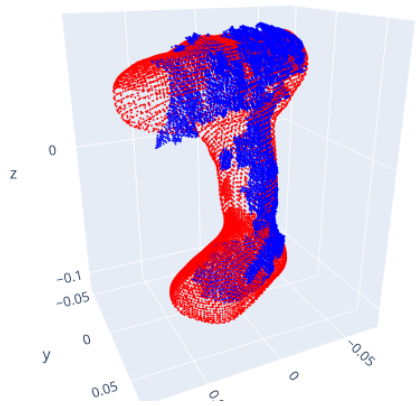

Fig. 15: ICP alignment for partial view 0 of the drill.

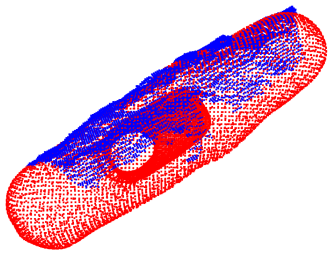

Fig. 16: ICP alignment for partial view 1 of the drill.

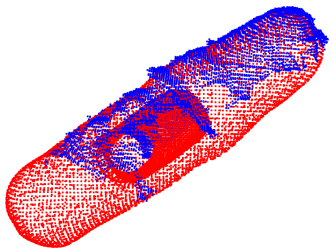

Fig. 17: ICP alignment for partial view 2 of the drill.

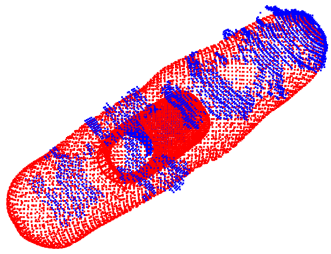

Fig. 18: ICP alignment for partial view 3 of the drill.

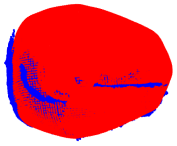

Fig. 19: ICP alignment for partial view 0 of the liquid container.

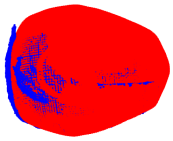

Fig. 20: ICP alignment for partial view 1 of the liquid container.

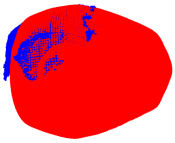

Fig. 21: ICP alignment for partial view 2 of the liquid container.

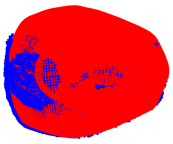

Fig. 22: ICP alignment for partial view 3 of the liquid container.